From Symbolic Agents to LLM-Powered Autonomy: A Comprehensive Survey on the Evolution of AI Agents

AI-Assisted Survey

April 2026

Abstract

The development of autonomous agents has been a central pursuit of artificial intelligence for over six decades. This survey provides a comprehensive account of the evolution of AI agents, tracing their intellectual lineage from symbolic planning systems and the Belief-Desire-Intention (BDI) model of the 1990s, through reinforcement learning agents of the 2000s–2010s, to the current paradigm of large language model (LLM)-based agents. We present a structured analysis of the core capabilities that define modern agents—reasoning, planning, memory, and tool use—examining foundational techniques including chain-of-thought prompting, ReAct, Tree of Thoughts, and Reflexion. We survey the major agent frameworks that have emerged since 2023, from early autonomous projects such as AutoGPT and BabyAGI to production-grade systems including LangChain, AutoGen, and CrewAI. We further examine the resurgence of multi-agent systems, where LLM-powered agents collaborate through natural language to tackle complex tasks in software development, scientific research, and web automation. Finally, we identify key open challenges—including reliability, safety, evaluation, and efficiency—and outline promising future directions such as learning from experience, multimodal agency, and human-agent collaboration. By connecting historical foundations with cutting-edge developments, this survey offers both newcomers and experienced researchers a unified perspective on where the field has been, where it stands, and where it is headed.

Keywords: AI agents, large language models, autonomous agents, multi-agent systems, tool use, planning, memory

1 Introduction

The concept of an autonomous agent—an entity capable of perceiving its environment, reasoning about goals, and taking actions to achieve them—has been a central aspiration of artificial intelligence since its inception. From the early rule-based expert systems of the 1970s to the reinforcement learning breakthroughs of the 2010s, the pursuit of intelligent agency has driven some of the most significant advances in the field. However, the emergence of large language models (LLMs) such as GPT-3 [2] and its instruction-tuned successors [17] has fundamentally transformed the landscape of agent research, enabling a new generation of systems that can understand natural language instructions, reason about complex tasks, and interact with diverse tools and environments.

The rapid proliferation of LLM-based agents since 2023 has been remarkable. Projects such as AutoGPT [25] and BabyAGI [16] captured widespread attention by demonstrating that LLMs could be orchestrated to autonomously decompose goals, execute multi-step plans, and self-correct based on

feedback. Concurrently, academic research has produced foundational frameworks for agent reasoning, including ReAct [37], Reflexion [24], and Tree of Thoughts [38], which formalize how language models can synergize reasoning with action-taking. These developments have catalyzed an explosion of research spanning single-agent architectures, multi-agent collaboration, tool use, memory systems, and real-world applications.

Despite this rapid progress, the field lacks a unified narrative that traces the evolution of AI agents from their symbolic roots to the current LLM-powered paradigm. Existing surveys have provided valuable snapshots of specific aspects: Wang et al. [30] offered a comprehensive taxonomy of LLM-based autonomous agents, Xi et al. [35] examined the rise and potential of such systems, and Luo et al. [13] presented a methodology-centered analysis. However, few works have attempted to connect the historical foundations of agent research with the modern LLM-driven revolution in a single, coherent account.

This survey aims to fill that gap by providing a comprehensive overview of the evolution of AI agents, organized along both historical and thematic dimensions. Our contributions are as follows:

- We trace the intellectual lineage of AI agents from symbolic AI and the Belief-Desire-Intention (BDI) model through reinforcement learning to the current LLM-based paradigm.
- We provide a structured analysis of the core capabilities that define modern LLM-based agents: reasoning, planning, memory, and tool use.
- We survey the major agent frameworks and multi-agent systems that have emerged since 2023, analyzing their architectural choices and trade-offs.
- We identify key applications, open challenges, and promising future directions for the field.

The remainder of this paper is organized as follows. Sec. 2 establishes foundational definitions and a conceptual framework for understanding agents. Sec. 3 reviews the pre-LLM history of agent research. Sec. 4 examines the core techniques underlying LLM-based agents. Sec. 5 surveys major agent frameworks and toolkits. Sec. 6 discusses multi-agent systems. Sec. 7 presents applications and case studies. Sec. 8 identifies challenges and future directions, and Sec. 9 concludes.

2 Background and Definitions

2.1 What Is an Agent?

The term "agent" has been used with varying definitions across different communities. In the classical AI literature, Wooldridge and Jennings [33] defined an agent as a computer system situated in some environment that is capable of autonomous action in order to meet its design objectives. Russell and Norvig further refined this by characterizing an agent as anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

In the context of LLM-based systems, we adopt a working definition that synthesizes these perspectives: an **AI agent** is a system that uses a language model as its core reasoning engine, augmented with the ability to perceive its environment, maintain internal state, make plans, and take actions—including the use of external tools—to accomplish goals specified in natural language.

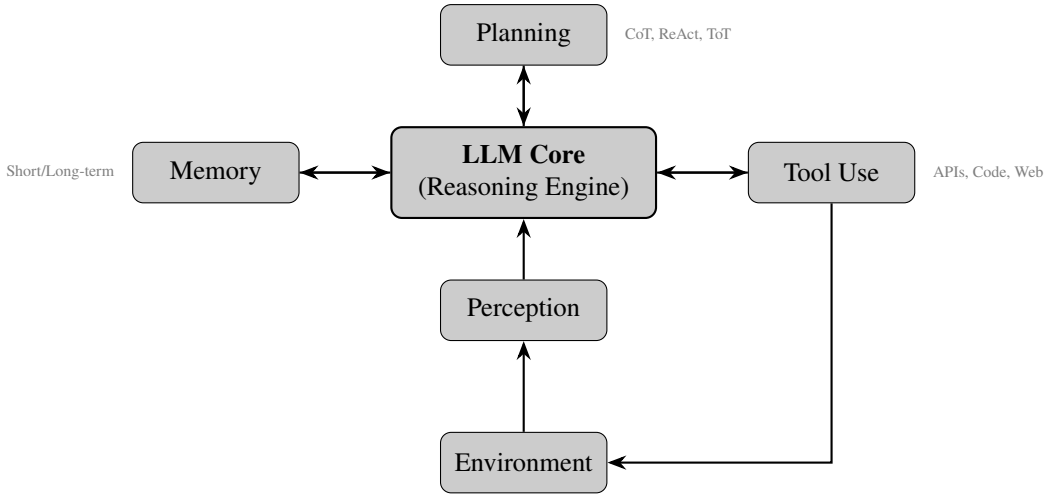

Figure 1: Architecture of a typical LLM-based agent. The LLM serves as the central reasoning engine, interacting with four core modules: planning (task decomposition and strategy), memory (experience storage and retrieval), tool use (external API and environment interaction), and perception (input processing).

2.2 Core Components of Modern Agents

Following the taxonomies proposed by Wang et al. [30] and Xi et al. [35], we identify four core components that characterize modern LLM-based agents:

1. **Perception**: The ability to receive and interpret inputs from the environment, including text, images, code, and structured data.
2. **Reasoning and Planning**: The capacity to decompose complex goals into sub-tasks, evaluate alternative strategies, and construct executable plans. This encompasses techniques such as chain-of-thought prompting [32], tree search [38], and self-reflection [24].
3. **Memory**: Mechanisms for storing and retrieving information across interactions, including short-term working memory (conversation context), long-term episodic memory (past experiences), and semantic memory (factual knowledge).
4. **Action and Tool Use**: The ability to execute actions in the environment, including calling APIs, running code, browsing the web, and manipulating files. Toolformer [22] and ToolLLM [20] demonstrated that LLMs can learn to use tools effectively.

2.3 Taxonomy of Agent Architectures

Agent architectures can be broadly classified along several dimensions:

- **Single-agent vs. Multi-agent**: Whether the system consists of one agent or multiple collaborating agents.
- **Reactive vs. Deliberative**: Whether the agent responds directly to stimuli or engages in explicit planning and reasoning.
- **Monolithic vs. Modular**: Whether capabilities are integrated into a single model or distributed across specialized components.
- **Static vs. Adaptive**: Whether the agent's behavior is fixed or evolves through learning and self-improvement.

3 Early Foundations of AI Agents

3.1 Symbolic AI and Expert Systems (1960s–1990s)

The earliest conceptions of artificial agents emerged from the symbolic AI tradition. Systems like STRIPS (Stanford Research Institute Problem Solver, 1971) introduced formal planning as a core agent capability, representing the world as a set of logical predicates and searching for action sequences that transform an initial state into a goal state. The General Problem Solver (GPS) by Newell and Simon (1963) embodied the idea that intelligence could be achieved through symbol manipulation and means-ends analysis.

Expert systems of the 1980s, such as MYCIN and DENDRAL, demonstrated that domain-specific knowledge could be encoded as production rules to create systems capable of reasoning and decision-making within narrow domains. While these systems lacked the generality and adaptability of modern agents, they established fundamental principles: the separation of knowledge from inference, the importance of explanation capabilities, and the value of modular knowledge representation.

3.2 The BDI Model and Multi-Agent Systems (1990s)

The Belief-Desire-Intention (BDI) model, formalized by Rao and Georgeff [21], provided a philosophical and computational framework for rational agency. In this model, agents maintain beliefs about the world, desires representing their goals, and intentions that commit them to particular courses of action. The BDI architecture influenced a generation of agent platforms, including JACK, Jason, and Jadex, and remains relevant as a conceptual foundation for understanding modern agent behavior.

Concurrently, the multi-agent systems (MAS) community developed frameworks for agent communication, coordination, and negotiation. Standards such as FIPA-ACL (Foundation for Intelligent Physical Agents – Agent Communication Language) established protocols for inter-agent messaging, while research on mechanism design and game theory provided formal tools for analyzing strategic interactions among self-interested agents.

Wooldridge and Jennings [33] provided an influential synthesis of agent theory and practice, identifying key properties of intelligent agents: autonomy, social ability, reactivity, and pro-activeness. These properties continue to serve as design criteria for modern agent systems.

3.3 Reinforcement Learning Agents (2000s–2010s)

The reinforcement learning (RL) paradigm, comprehensively described by Sutton and Barto [28], offered a fundamentally different approach to agent design. Rather than encoding knowledge explicitly, RL agents learn optimal behavior through trial-and-error interaction with their environment, guided by reward signals.

Key milestones in RL-based agency include: Deep Q-Networks (DQN) [15] achieving human-level performance on Atari games; AlphaGo [26] defeating the world champion in Go through a combination of deep learning and Monte Carlo tree search; and OpenAI Five (2019) demonstrating coordinated multi-agent play in the complex real-time strategy game Dota 2.

While RL agents achieved remarkable performance in specific domains, they faced significant limitations: sample inefficiency requiring millions of interactions, difficulty in transferring learned policies across tasks, and the challenge of specifying reward functions for open-ended real-world tasks.

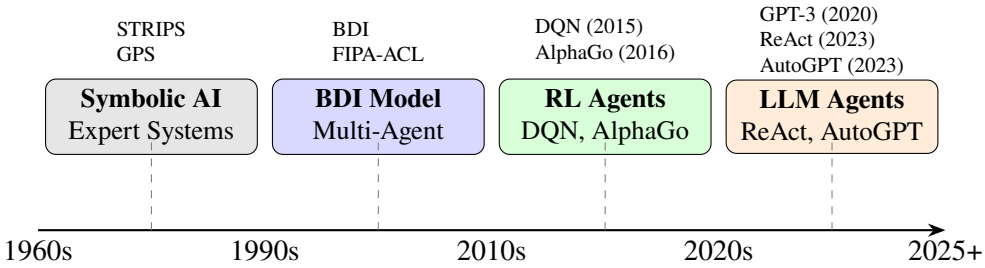

Figure 2: Timeline of AI agent evolution from symbolic systems (1960s–1990s) through BDI and multi-agent systems (1990s), reinforcement learning agents (2010s), to the current LLM-based paradigm (2020s–present).

These limitations set the stage for the LLM-based agent paradigm, which leverages the broad world knowledge and language understanding of pre-trained models to overcome many of these challenges.

4 LLM-Based Agents

The emergence of large language models with strong instruction-following and reasoning capabilities has enabled a fundamentally new approach to building autonomous agents. Rather than learning from scratch through environment interaction, LLM-based agents leverage the vast knowledge encoded during pre-training to reason about tasks, generate plans, and interact with tools—all through natural language.

4.1 Reasoning and Prompting Strategies

The foundation of LLM-based agency lies in structured prompting strategies that elicit systematic reasoning from language models.

Chain-of-Thought (CoT). Wei et al. [32] demonstrated that prompting LLMs to generate intermediate reasoning steps dramatically improves performance on complex tasks. By including examples of step-by-step reasoning in the prompt, models learn to decompose problems and reason through them sequentially. This simple yet powerful technique laid the groundwork for more sophisticated agent reasoning.

ReAct. Yao et al. [37] introduced the ReAct framework, which interleaves reasoning traces with action execution. In ReAct, the agent alternates between generating thoughts (reasoning about what to do next) and taking actions (such as searching the web or querying a database), using the observations from actions to inform subsequent reasoning. This tight coupling of reasoning and acting proved highly effective for tasks requiring interaction with external environments.

Tree of Thoughts (ToT). Yao et al. [38] extended chain-of-thought reasoning into a tree-structured search process. Rather than following a single reasoning chain, ToT explores multiple reasoning paths simultaneously, evaluates intermediate states, and backtracks when necessary. This deliberate problem-solving approach enables LLMs to tackle tasks that require exploration and strategic planning.

Self-Consistency. Wang et al. [31] proposed self-consistency decoding, which samples multiple reasoning paths and selects the most consistent answer through majority voting. This approach improves

robustness over single-chain reasoning and has become a standard technique in agent systems that require reliable decision-making.

Reflexion. Shinn et al. [24] proposed a framework for verbal reinforcement learning, where agents reflect on their failures and generate natural language feedback to improve future attempts. Unlike traditional RL, which requires scalar reward signals, Reflexion leverages the language model's ability to articulate what went wrong and how to improve, creating a self-improvement loop without parameter updates.

Sumers et al. [27] provided a unifying perspective on these techniques through the lens of cognitive architectures, proposing a framework that maps LLM agent designs to established cognitive science concepts including perception, memory, decision-making, and metacognition.

4.2 Memory Systems

Effective agency requires the ability to store and retrieve information across interactions. Park et al. [18] demonstrated the power of memory in their Generative Agents work, where simulated agents maintained detailed memories of their experiences, reflected on them to form higher-level abstractions, and used these memories to guide future behavior. Their architecture included three key components:

- **Memory stream**: A comprehensive record of the agent's experiences, stored as natural language descriptions with timestamps.
- **Retrieval**: A mechanism that surfaces relevant memories based on recency, importance, and relevance to the current context.
- **Reflection**: A process that periodically synthesizes recent memories into higher-level insights and generalizations.

This memory architecture has influenced numerous subsequent systems and remains a reference design for agent memory.

4.3 Tool Use and Environment Interaction

A defining characteristic of LLM-based agents is their ability to use external tools to extend their capabilities beyond pure text generation.

Schick et al. [22] showed that language models can learn to use tools through self-supervised training, inserting API calls into text when doing so improves prediction accuracy. Qin et al. [20] scaled this approach to over 16,000 real-world APIs, demonstrating that LLMs can navigate complex API ecosystems to accomplish diverse tasks. Shen et al. [23] introduced HuggingGPT, which uses ChatGPT as a controller to coordinate specialized AI models from Hugging Face, demonstrating a task planning → model selection → execution → response generation pipeline that extends agent capabilities far beyond text processing.

Modern agent systems typically provide tool access through structured interfaces: the agent generates a tool call in a specified format (e.g., JSON), the system executes the call and returns the result, and the agent incorporates the result into its reasoning. This pattern has been standardized in frameworks such as OpenAI's function calling and Anthropic's tool use protocols.

4.4 Self-Improvement and Iterative Refinement

Madaan et al. [14] introduced Self-Refine, a framework where LLMs iteratively improve their own outputs through self-generated feedback. The process involves three steps: initial generation, self-critique (identifying specific issues), and refinement (addressing the identified issues). This iterative approach has proven effective across diverse tasks including code generation, mathematical reasoning, and creative writing.

5 Agent Frameworks and Toolkits

The rapid growth of LLM-based agent research has been accompanied by the development of numerous frameworks and toolkits that provide reusable infrastructure for building agent systems.

5.1 Early Autonomous Agent Projects

AutoGPT. Launched in March 2023, AutoGPT [25] was among the first projects to demonstrate fully autonomous LLM-based agency. Given a high-level goal, AutoGPT uses GPT-4 to decompose the goal into sub-tasks, execute them using tools (web search, file operations, code execution), and iteratively refine its approach based on results. Despite limitations in reliability and tendency toward infinite loops, AutoGPT captured public imagination and inspired a wave of autonomous agent projects.

BabyAGI. Nakajima's BabyAGI [16] introduced a minimalist architecture for task-driven autonomous agents. The system maintains a task list, uses an LLM to prioritize and decompose tasks, executes them sequentially, and generates new tasks based on results. Its simplicity made it highly influential as a conceptual template for agent architectures.

5.2 Production-Grade Frameworks

LangChain. LangChain [3] emerged as the dominant framework for building LLM applications, providing abstractions for chains (sequential LLM calls), agents (LLM-driven decision-making with tool use), and memory (conversation and long-term storage). Its agent module supports multiple reasoning strategies including ReAct, plan-and-execute, and custom agent types. LangGraph, an extension of LangChain, introduced graph-based workflows for more complex agent orchestration.

AutoGen. Microsoft's AutoGen [34] pioneered the concept of multi-agent conversation, where multiple LLM-powered agents collaborate through structured dialogue. AutoGen provides abstractions for conversable agents, group chat managers, and human-in-the-loop interactions, making it particularly suited for complex workflows that benefit from diverse perspectives and iterative refinement.

CrewAI. CrewAI [5] introduced a role-based approach to multi-agent orchestration, where agents are assigned specific roles (e.g., researcher, writer, reviewer) and collaborate on tasks through a structured workflow. Its emphasis on role specialization and process management has made it popular for business automation applications.

Table 1: Comparison of major LLM-based agent frameworks. "Multi-Agent" indicates native support for multi-agent collaboration. "Human-in-Loop" indicates built-in mechanisms for human oversight.

Framework	Year	Multi-Agent	Tool Use	Human-in-Loop	Key Feature
AutoGPT	2023	No	Yes	No	Autonomous goal pursuit
BabyAGI	2023	No	Yes	No	Task-driven loop
LangChain	2022	Yes	Yes	Yes	Composable chains
AutoGen	2023	Yes	Yes	Yes	Multi-agent conversation
CrewAI	2024	Yes	Yes	Yes	Role-based orchestration
MetaGPT	2023	Yes	Yes	No	SOP-driven collaboration

Table 2: Major benchmarks for evaluating LLM-based agents. Benchmarks vary in scope, environment type, and the agent capabilities they assess.

Benchmark	Domain	Capabilities Tested	Year
AgentBench	Multi-domain (8 envs)	Reasoning, tool use, coding	2023
WebArena	Web navigation	Planning, grounding, action	2023
SWE-bench	Software engineering	Code understanding, patching	2024
ToolBench	API usage (16k+ APIs)	Tool selection, composition	2023
AgentBoard	Multi-turn dialogue	Multi-step reasoning	2024

5.3 Evaluation and Benchmarking

The proliferation of agent frameworks has created a pressing need for standardized evaluation. Liu et al. [12] introduced AgentBench, a comprehensive benchmark for evaluating LLMs as agents across eight distinct environments including operating systems, databases, and web browsing. Liu et al. [11] proposed AgentBoard, providing analytical evaluation of multi-turn agent interactions. Duan et al. [6] conducted comparative studies of LangGraph and CrewAI in multi-agent settings. Table 2 summarizes the major evaluation benchmarks for LLM-based agents.

6 Multi-Agent Systems

While single-agent systems have demonstrated impressive capabilities, many real-world tasks benefit from the collaboration of multiple specialized agents. The LLM era has revitalized interest in multi-agent systems, with new approaches that leverage natural language as the primary medium of inter-agent communication. Guo et al. [7] provided a comprehensive survey of LLM-based multi-agent systems, categorizing them by communication topology, coordination mechanism, and application domain.

6.1 Role-Playing and Debate

Li et al. [10] introduced CAMEL (Communicative Agents for "Mind" Exploration), a framework where two LLM agents engage in role-playing conversations to collaboratively solve tasks. By assigning complementary roles (e.g., AI user and AI assistant), CAMEL demonstrated that structured dialogue between agents could produce more thorough and creative solutions than single-agent approaches.

6.2 Software Development as Multi-Agent Collaboration

Software development has emerged as a compelling application domain for multi-agent systems, as it naturally decomposes into specialized roles.

Hong et al. [8] proposed MetaGPT, which assigns agents roles mirroring a software company (product manager, architect, engineer, QA) and coordinates their interactions through standardized operating procedures (SOPs). By encoding human workflow knowledge into the agent coordination protocol, MetaGPT produces more coherent and higher-quality software artifacts than unstructured multi-agent approaches.

Qian et al. [19] developed ChatDev, a virtual software company where LLM agents collaborate through a chat-based workflow spanning design, coding, testing, and documentation phases. ChatDev demonstrated that multi-agent collaboration could produce complete, functional software from natural language specifications.

6.3 Coordination Mechanisms

Multi-agent LLM systems employ various coordination mechanisms:

- **Sequential pipeline**: Agents process tasks in a fixed order, each building on the previous agent's output.
- **Debate and consensus**: Agents propose solutions, critique each other's proposals, and converge on a consensus through structured argumentation.
- **Hierarchical delegation**: A manager agent decomposes tasks and delegates sub-tasks to specialized worker agents.
- **Blackboard architecture**: Agents share information through a common workspace, contributing and consuming information asynchronously.

Wu et al. [34] formalized the multi-agent conversation paradigm, where agents interact through natural language messages in group chat settings, with configurable turn-taking and termination conditions.

7 Applications and Case Studies

LLM-based agents have been deployed across a wide range of application domains, demonstrating their versatility and practical impact.

7.1 Software Engineering

Software engineering has been one of the most successful application areas for LLM-based agents. Chen et al. [4] established the foundation with Codex, demonstrating that LLMs trained on code could generate functionally correct programs from natural language descriptions. Yang et al. [36] introduced SWE-agent, which autonomously resolves GitHub issues by navigating codebases, understanding bug reports, and generating patches. SWE-agent achieved state-of-the-art performance on the SWE-bench benchmark, resolving a significant fraction of real-world software issues without human intervention.

7.2 Scientific Research

Agents are increasingly being applied to accelerate scientific research workflows, including literature review, hypothesis generation, experiment design, and paper writing. Boiko et al. [1] demonstrated that LLM agents equipped with domain-specific tools can autonomously plan and execute chemical experiments, achieving results published in *Nature*. Wang et al. [29] introduced Voyager, an embodied lifelong learning agent in Minecraft that continuously explores, acquires diverse skills, and makes novel discoveries without human intervention—demonstrating key principles of open-ended agent learning. In the biomedical domain, agents have been deployed for drug discovery, clinical trial design, and medical literature synthesis [9].

7.3 Web Navigation and Task Automation

Web-based agents represent a natural application of the perception-reasoning-action loop. Zhou et al. [39] introduced WebArena, a realistic and reproducible web environment for benchmarking autonomous agents on tasks such as online shopping, information retrieval, and content management. Systems such as Mind2Web have further established benchmarks for evaluating agents that navigate real websites. These agents must handle the complexity of dynamic web interfaces, including JavaScript-rendered content, authentication flows, and multi-step transactions.

7.4 Personal Assistants and Productivity

LLM-based agents are being integrated into productivity tools as intelligent assistants that can manage calendars, draft emails, organize files, and coordinate workflows. Unlike traditional automation tools that follow rigid scripts, agent-based assistants can interpret ambiguous instructions, adapt to changing contexts, and handle exceptions through reasoning.

8 Challenges and Future Directions

Despite remarkable progress, LLM-based agents face several fundamental challenges that must be addressed for the field to mature.

8.1 Reliability and Robustness

Current LLM-based agents suffer from several reliability issues: hallucination (generating plausible but incorrect information), error propagation (where early mistakes compound through multi-step reasoning), and inconsistent behavior across runs. Improving reliability requires advances in uncertainty quantification, error detection, and graceful degradation.

8.2 Safety and Alignment

As agents gain the ability to take consequential actions in the real world—executing code, sending messages, making purchases—ensuring their alignment with human intentions becomes critical. Key concerns include: preventing agents from taking harmful actions, ensuring they respect boundaries and

permissions, maintaining transparency about their reasoning and actions, and handling adversarial inputs that attempt to manipulate agent behavior (prompt injection).

8.3 Evaluation and Benchmarking

Evaluating agent systems remains challenging due to the open-ended nature of many agent tasks, the difficulty of measuring partial progress, and the gap between benchmark performance and real-world utility [11, 12]. Future evaluation frameworks must capture not only task completion rates but also efficiency, safety, user satisfaction, and robustness to distribution shift.

8.4 Scalability and Efficiency

LLM-based agents incur significant computational costs due to repeated model inference, long context windows, and multi-turn interactions. Reducing these costs through more efficient architectures, caching strategies, and selective reasoning (knowing when to think carefully vs. act quickly) is essential for practical deployment.

8.5 Future Directions

Several promising research directions are emerging:

- **Learning from experience**: Agents that improve their performance over time by learning from past successes and failures, potentially through fine-tuning or in-context learning.
- **Multimodal agents**: Extending agent capabilities beyond text to include vision, audio, and physical interaction, enabling embodied agents that operate in the real world.
- **Agent-computer interfaces**: Designing better interfaces between agents and computing environments, as explored by SWE-agent [36], to improve agent effectiveness.
- **Formal verification**: Developing methods to formally verify agent behavior and provide guarantees about safety and correctness.
- **Human-agent collaboration**: Moving beyond fully autonomous agents toward systems that effectively collaborate with humans, combining the strengths of both.

9 Conclusion

This survey has traced the evolution of AI agents from their symbolic roots in the 1960s through the reinforcement learning era to the current LLM-powered paradigm. The emergence of large language models has fundamentally transformed agent research by providing a universal interface for reasoning, planning, and action through natural language.

We have examined the core techniques that enable modern LLM-based agents—structured prompting, memory systems, tool use, and self-improvement—and surveyed the major frameworks and multi-agent systems that have emerged since 2023. The field has progressed rapidly from proof-of-concept demonstrations like AutoGPT to production-grade frameworks like LangChain, AutoGen, and CrewAI, and has found compelling applications in software engineering, scientific research, and task automation.

However, significant challenges remain. Reliability, safety, evaluation, and efficiency are all active areas of research that must be addressed for LLM-based agents to achieve their full potential. The path

forward likely involves a combination of better architectures, more robust evaluation frameworks, improved human-agent collaboration paradigms, and advances in the underlying language models themselves.

As LLMs continue to improve in capability and efficiency, and as the ecosystem of tools, frameworks, and benchmarks matures, we anticipate that AI agents will become increasingly central to how humans interact with and leverage artificial intelligence. The journey from STRIPS to SWE-agent represents not just a technological evolution, but a fundamental shift in our conception of what artificial intelligence can achieve.

References

[1] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[3] Harrison Chase. Langchain: Building applications with llms through composability. *GitHub repository*, 2022.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.

[5] CrewAI Inc. Crewai: Framework for orchestrating role-playing autonomous ai agents. https://github.com/joaomdmoura/crewAI, 2024. Accessed: 2024-04-22.

[6] Zhihua Duan et al. Exploration of llm multi-agent application based on langgraph and crewai. *arXiv preprint arXiv:2411.18241*, 2024. doi: 10.48550/arXiv.2411.18241.

[7] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024. doi: 10.48550/arXiv.2402.01680.

[8] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Zhang, Ceyao Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. doi: 10.48550/arXiv.2308.00352.

[9] Kexin Huang, Ege Altay, Mert Karabacak, and Ioana Danciu. Benchmarking large language model agents on conversational medicine. *arXiv preprint arXiv:2402.08043*, 2024. doi: 10.48550/arXiv.2402.08043.

[10] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2303.17760.

[11] Chang Liu, Qianli Liu, Yao Huang, Yao Zheng, Yiming Gao, Zihao Zhao, Ziming Zhu, Huanyu Jiang, Bolin Ding, and Quanshi Gu. Agentboard: An analytical evaluation board of multi-turn llm agents. *arXiv preprint arXiv:2401.13178*, 2024. doi: 10.48550/arXiv.2401.13178.

[12] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023. doi: 10.48550/arXiv.2308.03688.

[13] Jiaqi Luo et al. Large language model agent: A survey on methodology, application and evaluation. *arXiv preprint arXiv:2503.21460*, 2025. doi: 10.48550/arXiv.2503.21460.

[14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024. doi: 10.48550/arXiv.2303.17651.

[15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[16] Yohei Nakajima. Babyagi. https://github.com/yoheinakajima/babyagi, 2023. Accessed: 2024-04-22.

[17] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

[18] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. doi: 10.1145/3586183.3606763.

[19] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023. doi: 10.48550/arXiv.2307.07924.

[20] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023. doi: 10.48550/arXiv.2307.16789.

[21] Anand S Rao and Michael P Georgeff. Bdi agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, pages 312–319, 1995.

[22] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024. doi: 10.48550/arXiv.2302.04761.

[23] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2303.17580.

[24] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024. doi: 10.48550/arXiv.2303.11366.

[25] Significant Gravitas. Autogpt: An autonomous gpt-4 experiment. https://github.com/Significant-Gravitas/AutoGPT, 2023. Accessed: 2024-04-22.

[26] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[27] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2024. doi: 10.48550/arXiv.2309.02427.

[28] Richard S Sutton and Andrew G Barto. Reinforcement learning: An introduction. 2018.

[29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. doi: 10.48550/arXiv.2305.16291.

[30] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.

[31] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2023. doi: 10.48550/arXiv.2203.11171.

[32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. doi: 10.48550/arXiv.2201.11903.

[33] Michael Wooldridge and Nicholas R Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995. doi: 10.1017/S0269888900008122.

[34] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. doi: 10.48550/arXiv.2308.08155.

[35] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023. doi: 10.48550/arXiv.2309.07864.

[36] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. doi: 10.48550/arXiv.2405.15793.

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.03629.

[38] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2024. doi: 10.48550/arXiv.2305.10601.

[39] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. doi: 10.48550/arXiv.2307.13854.